



5.3.6 LU factorization with complete pivoting

LU factorization with partial pivoting builds on the insight that pivoting (rearranging) rows in a linear system does not change the solution: if $Ax = b$ then $P(p)Ax = P(p)b$, where p is a pivot vector. Now, if r is another pivot vector, then notice that $P(r)^T P(r) = I$ (a simple property of pivot matrices) and $AP(r)^T$ permutes the columns of A in exactly the same order as $P(r)A$ permutes the rows of A .

What this means is that if $Ax = b$ then $P(p)AP(r)^T(P(r)x) = P(p)b$. This supports the idea that one might want to not only permute rows of A , as in partial pivoting, but also columns of A . This is done in a variation on LU factorization that is known as LU factorization with *complete pivoting*.

The idea is as follows: Given matrix A , partition

$$A = \begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix}.$$

Now, instead of finding the largest element in magnitude in the first column, find the largest element in magnitude in the entire matrix. Let's say it is element (π_1, ρ_1) . Then, one permutes

$$\begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} := P(\pi_1) \begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} P(\rho_1)^T,$$

making α_{11} the largest element in magnitude. We will later see that the magnitude of α_{11} impacts *element growth* in the remaining matrix (A_{22}) and that in turn impacts the numerical stability (accuracy) of the algorithm. By choosing α_{11} to be as large as possible in magnitude, the magnitude of multipliers is reduced as is element growth.

The problem is that complete pivoting requires $O(n^2)$ comparisons per iteration. Thus, the number of comparisons is of the same order as the number of floating point operations. Worse, it completely destroys the ability to cast most computation in terms of matrix-matrix multiplication, thus impacting the ability to attain much greater performance.

In practice LU with complete pivoting is not used.